Capítulo 1. Deprogramador a ingeniero

Este trabajo se ha traducido utilizando IA. Agradecemos tus opiniones y comentarios: translation-feedback@oreilly.com

> *Las habilidades fundamentales, siempre tediosas de aprender, parecen obsoletas. Y podrían serlo, si existiera un atajo para convertirse en experto. Pero el camino hacia la experiencia requiere una base sólida en los hechos.*
>
> —Ethan Mollick, de *Co-Intelligence* (Portfolio, 2024)

Ser ingeniero de software requiere una amplia gama de habilidades en diversas áreas. Comprender lo que el cliente realmente pide. Traducir esas necesidades en código mantenible. Escribir pruebas para garantizar que el software hace lo que tú crees que debe hacer. Crear interfaces de usuario que funcionen. Arquitectura. Trabajar con datos. Llevar el código a producción. Si quieres crecer profesionalmente como profesional del software, debes centrarte en algo más que escribir código. Debes abarcar todo el arte de la ingeniería de software.

Para pasar de programador a ingeniero, debes dominar las habilidades fundamentales del ciclo de vida del desarrollo de software (SDLC), trabajar de forma inteligente, reconocer lo que no sabes y descubrir cómo subsanar esas lagunas. En este capítulo, encontrarás algunos consejos y recomendaciones para empezar. Aprenderás las diferentes vías para convertirte en ingeniero, así como los conocimientos clave que esos enfoques suelen omitir. En definitiva, este capítulo te ofrecerá consejos muy valiosos que te ayudarán en tu camino y allanarán el terreno para pasar de programador a ingeniero.

Pero primero, hablemos de lo que significa ser ingeniero de software.

Un ingeniero con cualquier otro nombre

El software está lleno de términos sobrecargados (a menudo reutilizados), lo que deja un amplio margen para la ambigüedad. Si le pides a cinco profesionales del software que definan una palabra, probablemente obtendrás al menos cinco respuestas. Originalmente, «ordenadores» se refería a los seres humanos que realizaban cálculos. Ahora, se oyen términos como programador, codificador, desarrollador e ingeniero de software. ¿Son sinónimos?

No, no lo son. *Programador* o *codificador* a menudo implica a alguien centrado en la tarea singular de generar, bueno, código.[1] En general, pueden estar muy cualificados en un lenguaje y/o marco específico, pero es posible que no comprendan todo el ciclo de vida del desarrollo

de software (SDLC). Aunque muchos comienzan su carrera como programadores o codificadores, el camino hacia los ascensos requiere ir más allá de la simple corrección de errores o la implementación de funciones aleatorias.

L*os desarrolladores* suelen tener una mejor comprensión del panorama general de la entrega de software. Por lo general, conocen varios lenguajes y marcos y tienen experiencia en múltiples ámbitos empresariales. A menudo, con más experiencia que un programador o codificador, han comenzado a explorar más el SDLC y pueden servir de mentores o compañeros para los miembros del equipo con menos experiencia.

Pero pasar al ámbito de la ingeniería de software requiere que apliques *la disciplina de la ingeniería* a todo el SDLC. Como *ingeniero de software*, se da por sentado que eres competente en la escritura de código, pero se espera que pienses en la escalabilidad, la fiabilidad, la eficiencia y la seguridad. Has ido más allá de la comprensión superficial de los algoritmos y el diseño. Conoces no solo las reglas, sino también cuándo deben flexibilizarse e incluso cuándo deben romperse. Los ingenieros de software se encargan de los sistemas más complejos y críticos que existen actualmente en producción.

Los bootcamps y las universidades suelen centrarse en los aspectos mecánicos de la escritura de código, creando personas versadas en programación. Crean desarrolladores o programadores. Ese es el punto de partida para una carrera en el mundo del software. Para destacar, ascender y trabajar en los proyectos más interesantes, debes ir más allá de la simple escritura de código.

El software es mucho más que crear programas sintácticamente correctos. Los conocimientos necesarios para ser una ingeniería de software de éxito hoy en día van más allá del aprendizaje de un lenguaje de programación; también debes ser un experto en el ciclo de vida completo de un producto de software. Este libro pretende ser tu guía en ese viaje, para mostrarte las cosas que quizás no sabes que no sabes.

Los fundamentos son importantes

Los fundamentos son importantes. Los atletas profesionales dedican la mayor parte de su tiempo a trabajar en las cosas que aprendieron cuando empezaron a practicar su deporte. Los golfistas se centran en la postura, el agarre y la alineación. Los jugadores de baloncesto se centran en las bandejas, los pases y los tiros libres. No dedican tanto tiempo a las cosas que se ven en los resúmenes de los partidos. Aunque los fundamentos pueden no ser tan emocionantes, sin una base sólida, nunca habrían alcanzado la cima de su profesión.

Como todas las profesiones, el software también tiene principios básicos esenciales. Tómate tu tiempo para dominarlos. Pasarás la mayor parte de tu carrera trabajando con código existente; aprender a leer el código que ha escrito otra persona y comprender rápidamente una nueva base de código es fundamental para tu éxito. Las bases de código desconocidas pueden resultar muy intimidantes; sin embargo, debes sentirte cómodo con la ambigüedad. Incluso después de muchos meses, no comprenderás todos los matices de una base de código, y no

es necesario que lo comprendas todo antes de realizar cambios en el código.

Aunque probablemente conozcas los fundamentos de la escritura de código, es posible que no estés tan familiarizado con cómo escribir código que simplifique las cosas para la persona que venga después de ti. La tecnología tampoco se detiene; debes ser capaz de aprender cosas nuevas. Muchos programadores se obsesionan con la última tecnología o la característica más novedosa del lenguaje, mientras ignoran las habilidades sociales atemporales que les ayudarán a conseguir el próximo ascenso. Los fundamentos pueden no generar tanto revuelo como la última moda, pero marcan la diferencia entre el estancamiento profesional y las oportunidades con mayor alcance y responsabilidades.

Los muchos caminos para convertirse en ingeniero de software

Hay varios caminos para convertirse en ingeniería de software. Algunas personas obtienen una licenciatura en informática, mientras que otras asisten a campamentos de entrenamiento y otras son autodidactas. Muchas tienen títulos en campos relacionados, como la ingeniería eléctrica, y luego pasan a escribir código. Sinceramente, no importa cómo hayas aprendido a crear software.

Algunas personas pueden argumentar que su camino es «mejor», «más rápido» o «más barato». Cada enfoque tiene sus pros y sus contras. El autoaprendizaje no requiere el pago de matrículas y, dependiendo de

tu aptitud, puede llevarte unas semanas en lugar de años. En Internet no faltan vídeos y tutoriales; sin embargo, puede resultar abrumador navegar por ellos y requiere una gran motivación personal. El auto-aprendizaje también puede implicar una buena dosis de ensayo y error, ya que hay que trabajar con material de diferente calidad.

Fundamentalmente, los objetivos de los boot camps y los programas de grado son diferentes. Los programas de grado en informática están diseñados para prepararte para acceder a un programa de posgrado en informática. Se centran en algoritmos, diseño de lenguajes, teoría de compiladores, sistemas operativos y temas relacionados. Sin embargo, no te preparan necesariamente para trabajar día tras día en aplicaciones del mundo real. Los proyectos son pequeños, rara vez duran más de unos meses y casi siempre se empiezan desde cero, sin el lastre de las aplicaciones existentes.

Los boot camps condensan información muy específica en un breve periodo de tiempo, a menudo de unas pocas semanas. Tienden a centrarse más en el lenguaje del momento, los marcos de trabajo, etc. Se podría argumentar que este enfoque es más práctico, pero también se podría afirmar que los conocimientos son de naturaleza más transitoria, ya que la popularidad de los lenguajes y los marcos de trabajo fluctúa. Los boot camps a veces se imparten en universidades e incluso pueden ser impartidos por las mismas personas que imparten cursos de informática a nivel de grado, por lo que puede que ni siquiera haya diferencia en cuanto al profesor.

Cada uno de estos enfoques implica también un compromiso de tiempo diferente. Los boot camps suelen durar alrededor de 600 horas re-

partidas en varias semanas, y tienes acceso casi constante a alguien que puede ayudarte a superar las dificultades. Un título de asociado suele ser de 850 horas repartidas en uno o dos años, mientras que un programa de licenciatura es de alrededor de 1400 horas repartidas en cuatro o cinco años.

Algunos afirman que un título universitario te da una ventaja inicial, aunque tiende a equilibrarse con el tiempo. Por supuesto, los programas tradicionales de informática tal y como los conoces hoy en día son relativamente nuevos, pero no hace mucho tiempo la mayoría de los programadores eran matemáticos de formación.

NOTA

Los primeros programas de grado en informática solían dar mucha importancia a las matemáticas, y algunos eran esencialmente grados de matemáticas con un poco de informática añadida. De hecho, muchos de los profesores tenían títulos en matemáticas. A día de hoy, algunas personas siguen pensando que la aptitud para las matemáticas es un requisito para ser un buen ingeniero de software. No es así. La programación tiene que ver con la comunicación, y la mayor parte de tu trabajo consiste en comunicarte con otros desarrolladores, no con el ordenador. La aptitud para los idiomas es mucho más importante que la habilidad matemática. Como es lógico, escribir código tiene que ver con el lenguaje, no con los números, algo sobre lo que leerás en el capítulo 2.

En última instancia, el éxito en la industria del software se basa en la resolución de problemas, la experimentación y la creatividad. Si tienes la mentalidad adecuada, lo tienes. Punto. No importa cómo hayas

desarrollado tus habilidades, y no tienes que disculparte por tu trayectoria.

Lo que te enseñaron frente a lo que necesitas saber

Independientemente de tu trayectoria de aprendizaje, te enseñaron a escribir código. Aprendiste uno o dos lenguajes, aprendiste sobre `foo` y `bar` y otros nombres de variables genéricas. Aprendiste un poco sobre depuración, diseño eficiente de algoritmos y otros temas relacionados. Sin embargo, hay una gran cantidad de cosas importantes que no se tratan en los planes de estudio típicos ni se mencionan en el aprendizaje autodirigido. ¿Por qué? Por lo general, debido a la falta de tiempo, espacio o la suposición de que el conocimiento es algo dado.[2]

Lo que estos cursos y materiales de aprendizaje omiten puede tener un impacto crítico en tu éxito a largo plazo. Entre los temas que se omiten se incluyen cómo trabajar con otros, cómo leer código, cómo escribir código *legible* y cómo trabajar de manera eficaz con código heredado. Abordaremos estos temas, y muchos más, a lo largo del resto del libro.

La mayoría de los desarrolladores aprenden en proyectos nuevos, aquellos que comienzan desde cero y sin la deuda técnica de los códigos base existentes. Dadas las limitaciones de tiempo de los semestres y los planes de estudio de los bootcamps, en la mayoría de las situaciones educativas, empiezas desde cero para practicar una habilidad específica o reforzar un concepto. Los proyectos suelen ser muy peque-

ños y aislados, y a menudo se programa en solitario. Aunque eso funciona en un entorno educativo, no es representativo del mundo real.

Como ingeniero de software e , pasarás la mayor parte de tu tiempo trabajando con aplicaciones existentes, heredadas o antiguas, y con todo el bagaje de decisiones tomadas antes de tu llegada.[3] E incluso cuando tengas el lujo de contar con un nuevo código base, este acumulará deuda técnica en los meses siguientes. Los proyectos de software actuales consisten en cientos de miles o millones de líneas de código creadas por equipos de personas de todo el mundo. Trabajarás constantemente con otras personas, ya sean tus compañeros de equipo o tus clientes. Tu código no solo existirá en un repositorio para que alguien lo califique, sino que estará en producción, donde personas reales confiarán en que funcione según lo diseñado y aporte valor empresarial. Por decirlo suavemente, tu primer contacto con la realidad del software puede ser un shock.

En la escuela, la mayoría de los proyectos son cortos, de unos pocos meses, quizá menos. Sin embargo, en el mundo real, los proyectos nunca terminan realmente, aunque algún día puedan abandonarse, como el arte. El código pasa la mayor parte de su vida en la fase de mantenimiento, lo que es otra forma de decir que el software está lleno de productos con inversiones continuas, en contraposición a proyectos discretos que tienen un inicio y un final bien definidos. Mientras un producto de software siga aportando valor empresarial, habrá una inversión continua.

Adopta la filosofía del programador perezoso

La filosofía del programador perezoso no consiste en pasar el día viendo vídeos de gatos en tu red social favorita, ni es una metodología para eludir tu trabajo. *La filosofía del programador perezoso* es una filosofía centrada en la eficiencia. Muchos ingenieros de software noveles se precipitan instintivamente a buscar soluciones; empiezan a escribir código inmediatamente sin darse un respiro para considerar el ámbito del problema. Ser estratégicamente perezoso te da tiempo para pensar, lo que en última instancia te ayuda a ser más productivo y a crear mejores soluciones.

> *Tómate el tiempo necesario para ahorrar tiempo.*
>
> —Pat Parelli, reconocido entrenador de equitación (atribuido)

Ten cuidado con el enfoque de fuerza bruta « ». Puedes empezar por ahí, pero debes iterar tu diseño. Puede ser útil comprender conceptos como la notación O grande.[4] Por ejemplo, es posible que hayas codificado una rutina de ordenación por burbuja, pero también hayas aprendido que no es el mecanismo de ordenación más eficiente. Como ingeniero, debes considerar el mejor caso, el peor caso y el caso promedio, y elegir lo más adecuado a partir de ahí.

Lo más probable es que no seas, de hecho, la primera persona en resolver un problema concreto. Dedica algo de tiempo a buscar una solu-

ción o biblioteca existente. En muchos casos, 10 minutos de búsqueda pueden ahorrarte días de trabajo. Si te parece que debería haber una forma más fácil, probablemente la haya. Hablaremos más sobre esto en de «No reinventes la rueda».

LA TAREA DE LAS MAYÚSCULAS

Aquí Nate. Al principio de mi carrera, me asignaron una tarea muy sencilla: poner en mayúscula las direcciones de facturación de una base de datos para garantizar la coherencia con la oficina de correos a la hora de imprimir y enviar las facturas. Mi jefe técnico me explicó exactamente qué hacer y dónde hacerlo, mostrándome la rutina que debía modificar, así como la función (escrita por alguien que ya no estaba) que debía llamar para poner en mayúscula la dirección de facturación. Al explorar la función de mayúsculas, me pareció demasiado complicada. En mi opinión, solo debería haber tenido un argumento: la cadena que se iba a poner en mayúsculas. Pero esta función no tenía un solo argumento. Tenía más de una docena. Me llevaría horas entender esta cosa casera.

Me pareció que este era el tipo de problema que el lenguaje de programación debería resolver por nosotros. Investigué un poco y, efectivamente, el lenguaje tenía una función de mayúsculas incorporada, que utilicé. Avancemos rápidamente hasta la semana siguiente, a nuestra reunión de equipo: mientras discutíamos en la sala sobre en qué estábamos trabajando, mencioné la tarea de las mayúsculas. Uno de mis colegas miró en mi dirección y dijo: «Oh, ¿has utilizado la función de mayúsculas de Joe?». Le dije que no.

Todas las cabezas de la sala se giraron. Fue un momento de silencio sepulcral. Aunque yo era el desarrollador con menos experiencia de la sala, nadie más sabía que el lenguaje tenía una función de mayúsculas incorporada. Quizás la función se añadió al lenguaje después de que el desarrollador anterior escribiera la función; no lo sé. La cuestión es

que, si crees que debería haber una forma más fácil, tómate tu tiempo para buscarla. Incluso unas pocas horas investigando opciones y soluciones alternativas podrían ahorrarte días o semanas de esfuerzo.

El valor de una nueva perspectiva

> *La frase más peligrosa del lenguaje es: «Siempre lo hemos hecho así».*
>
> —Grace Hopper, pionera de la informática y oficial de la marina

Aunque quizá no tengas tantos años de experiencia y líneas de código en tu haber como tus compañeros más veteranos, recuerda que tienes una cualidad muy valiosa como nuevo desarrollador: una mirada fresca. Las personas nuevas, ya sea en el código base, en el equipo o en la industria del software, suelen ver los retos con una perspectiva imparcial, sin el lastre de decisiones históricas que han dejado de ser útiles. Esta nueva perspectiva puede dar lugar a soluciones innovadoras, identificar ineficiencias o cuestionar supuestos arraigados. Los recién llegados también pueden aportar entusiasmo a un proyecto en declive, lo que se traduce en una energía renovada y un mayor compromiso por parte del equipo.

ASÍ ES COMO SIEMPRE LO HEMOS HECHO

Aquí Nate. Hace años, ayudé a un cliente con un problema de rendimiento relacionado con la forma en que una aplicación procesaba los widgets en un trabajo por lotes nocturno. Cuando la empresa comenzó, solo tenía unos pocos clientes con unas pocas cajas de widgets, que se completaban fácilmente durante la noche. A medida que mi cliente tuvo más éxito, tuvo más clientes, y esos clientes tenían más widgets. Cuando trabajé con ellos, su ejecución por lotes nocturna ya no terminaba durante la noche.

Me senté con ellos y revisé la arquitectura. Después de un par de horas, les hice una pregunta sencilla: «¿El procesamiento del widget A tiene algo que ver con el procesamiento del widget B?». No, al final se realiza una agregación, pero los widgets individuales son independientes. Hice una propuesta sencilla: ¿podrían procesar esos widgets en paralelo? Implementar un conjunto de trabajadores que cojan el siguiente widget, lo procesen y pongan los resultados en una cola para agregarlos.

Hubo una pausa significativa. Sí. Sí, eso funcionaría. Aunque, por supuesto, quería que mi jefe pensara que tenía una visión especial, en realidad no era así. Simplemente no estaba limitado por la frase más peligrosa que se puede pronunciar en una organización: «Siempre lo hemos hecho así». El hecho de que siempre lo hayamos hecho así no significa que sea la forma correcta, y desde luego no significa que sea la mejor. El enfoque que alguien adoptó hace años podría haber sido una casualidad o simplemente la opción más conveniente. Es probable

que haya una forma mejor. No tengas miedo de hacer preguntas o de buscar otro enfoque.

Nunca subestimes el valor de una perspectiva diferente y no tengas miedo de preguntar por qué las cosas son como son. Si encuentras algo que te parece extraño, fuera de lugar o que simplemente no tiene sentido, pide a un compañero de equipo que te lo explique. En algunos casos, un enfoque poco convencional tendrá razones válidas, pero la mayoría de las veces, la gente simplemente ha copiado lo que había cuando llegó. Siempre que tus preguntas provengan de un deseo de aprender y no de un intento de avergonzar a los desarrolladores del pasado (y del presente), la mayoría de los ingenieros de software participarán de forma positiva en la discusión.

No busques soluciones demasiado rápido

Cuando te encuentres con un problema, resiste la tentación de buscar una solución demasiado rápido. En su lugar, dedica tiempo a comprender la causa raíz del problema. Date un poco de espacio para pensar. No tenemos suficientes oportunidades para pensar sin interrupciones. Es probable que tu agenda de trabajo esté llena de reuniones y que tu herramienta de mensajería corporativa te interrumpa varias veces al día. Entre la reunión diaria, el propietario del producto, tu gerente, el jefe de tu equipo y tu arquitecto, probablemente estés dando una ac-

tualización del estado cada dos horas. Tu lista de tareas pendientes no disminuye y los evaluadores encontraron otra serie de errores difíciles. A pesar de toda la presión sobre tu tiempo, es vital que te tomes el tiempo para pensar en soluciones y no te apresures a buscar una solución rápida.

Aunque las soluciones rápidas resuelven tus problemas inmediatos, a largo plazo pueden causar más problemas. Recurrir a una solución provisional a menudo crea más retos. Para abordar esto, ha habido un movimiento para detectar los problemas en una fase más temprana del ciclo de desarrollo, es decir, «desplazándose hacia la izquierda» o hacia el principio del calendario del proyecto. Los errores detectados en la producción son, con diferencia, los más caros de solucionar. Los clientes se han visto afectados, lo que podría costarle dinero a tu empresa. Los datos pueden corromperse, lo que puede ser difícil de conciliar. La solución del problema subyacente suele abarcar múltiples rutinas que requieren una refactorización complicada y, probablemente, la coordinación con otros equipos.

El desplazamiento hacia la izquierda no se limita a solucionar problemas, sino que, en última instancia, consiste en adelantar todo lo que tiene valor en el proceso. La programación en pareja convierte las revisiones de código, que antes se realizaban días (o semanas) después de finalizar el trabajo, en una actividad en tiempo real. Convertir las pruebas en una propuesta continua simplifica la resolución de problemas, ya que permite identificarlos más cerca del momento en que se crean. La integración continua alivia el dolor de los cambios de código conflictivos al fusionarlos temprano y con frecuencia y, cuando se combina con la entrega continua, garantiza que cualquier variabilidad

en los entornos se descubra antes de que afecte a un cliente. Incorporar la seguridad en todo el proceso de desarrollo reduce la probabilidad de vulnerabilidades catastróficas.

Es mucho más barato encontrar esos problemas en las pruebas. Cuando los errores se encuentran más cerca del momento en que se escribió el código, el trabajo es una prioridad para el equipo de desarrollo, lo que les ahorra tiempo a la hora de descifrar el código subyacente. Los almacenes de datos de prueba se pueden restaurar con un mínimo esfuerzo. Aunque el problema pueda ser complejo de resolver, es posible que haya menos presión de tiempo para solucionarlo rápidamente, ya que no afecta a ningún cliente. Así nacieron las pruebas de control de calidad (QA) y los ingenieros de QA. Resulta que es aún más barato si se detectan problemas durante el desarrollo, lo que respalda en parte el impulso del desarrollo basado en pruebas (véase el capítulo 5).

Pero, ¿y si nunca hubieras escrito el error en primer lugar? ¿Y si se hubiera resuelto en el diseño?

> *La mayoría de los grandes problemas que tenemos con el software son problemas de conceptos erróneos. No tenemos una idea clara de lo que estamos haciendo antes de hacerlo. Y luego, adelante, adelante, adelante, adelante y lo hacemos todo.*
>
> —Rich Hickey, discurso inaugural de Clojure Conj 2010

Problemas de conceptos erróneos Los conceptos erróneos ( ) son endémicos en el software, y los malentendidos son un reto común en el

desarrollo de software. Términos como «cliente» y «consumidor» pueden parecer intercambiables, hasta que el propietario de tu producto dice que ese enfoque funcionará para un cliente, pero no para un consumidor. Son cosas similares, pero distintas, y el matiz es importante. Por muchas pruebas que se hagan, ningún sistema de tipos encontrará jamás un problema de conceptos erróneos. Una solución técnica por sí sola no puede resolver estos problemas, ya que se derivan de una diferencia fundamental en la comprensión.[5]

En lugar de solucionar un problema creado por un malentendido, intenta evitarlo desde el principio. ¿Cómo? Céntrate en resolver problemas. ¿Sabes qué problema resuelve tu aplicación? ¿Entiendes el contexto y las limitaciones? ¿Cuáles son los problemas relacionados? Tómate tu tiempo para pensar detenidamente en el problema.

Los desarrolladores tienden a menudo a sobreingeniar una solución. Aunque no siempre es un problema, hay muchas formas de resolver un problema. Nunca pierdas de vista el problema que estás tratando de resolver y no te precipites a buscar soluciones. ¿Qué pide tu cliente? ¿Cuál es su necesidad real? Con las cicatrices de la experiencia, nunca querrás meterte en un callejón sin salida, y es fácil imaginar todo lo que *podrías* necesitar. Pero tampoco querrás crear un montón de código que luego tendrás que descartar. O peor aún, que se quede sin usar.

Puede ser difícil de conseguir, pero busca el «porqué», o la causa raíz detrás de la solicitud del cliente (véase el recuadro «Las cinco preguntas»). Haz preguntas. Habla con tus clientes; habla con los propietarios de tus productos. En muchos casos, un cliente pedirá una cosa, pero necesitará otra porque no la ve. Están atrapados en «siempre lo hemos

hecho así». A veces, tus clientes acudirán a ti con una solución muy específica, por ejemplo: «Necesitamos poder exportar esta tabla a una hoja de cálculo». Es tentador crear inmediatamente la función, pero un ingeniero experimentado pregunta primero cuál es la necesidad subyacente. A menudo existe una desconexión entre lo que tus clientes piden y lo que resuelve su problema empresarial.

LAS CINCO PREGUNTAS

Los cinco porqués, que tienen su origen en el Sistema de Producción Toyota, son un método de resolución de problemas que se esfuerza por descubrir la causa raíz de un problema repitiendo la pregunta «¿por qué?» cinco veces, cada vez dirigiendo el siguiente «por qué» a la respuesta anterior a «¿por qué?». Hacerlo te permite profundizar más allá del síntoma o problema superficial para descubrir el problema fundamental. Por ejemplo:

- Problema: El robot dejó de funcionar.
  - ¿Por qué? El circuito se sobrecargó y se fundió un fusible.
  - ¿Por qué? Los cojinetes no estaban suficientemente lubricados, por lo que se bloquearon.
  - ¿Por qué? La bomba de aceite no circulaba suficiente aceite.
  - ¿Por qué? La entrada de la bomba está obstruida con virutas de metal.
  - ¿Por qué? La bomba no tiene filtro.

Veamos un ejemplo relacionado con el software. Soy Nate. Hace años, me encontré con una página web en la que todas las etiquetas de cierre estaban en la línea siguiente. Aunque al intérprete no le importaba, resultaba visualmente molesto ver etiquetas de cierre al principio de cada línea. Le pregunté a un compañero *por qué* alguien había utilizado este enfoque. Me respondió que era debido a un error en una versión antigua del servidor web que utilizábamos. Vale, *¿y por qué* seguimos utilizando esa versión? Ah, no la utilizamos. Vale, *¿y por qué* no se ha corregido ese error? Ah, sí, se ha corregido. Vale, entonces, *¿por qué* sigo viendo este patrón? Bueno, probablemente se trate de una pá-

gina web antigua. Te dije que se había creado unos días antes. En ese momento, acordamos que debíamos informar a los desarrolladores de que dejaran de seguir ese patrón y añadimos una tarea de limpieza para reelaborar las páginas que aún utilizaban el enfoque obsoleto.

En muchos casos, la fuente principal de los requisitos será un superusuario del sistema antiguo. Si bien sus conocimientos pueden ser muy valiosos, también están influidos por lo que tienen actualmente.[6] En algunos casos, no entenderán el *porqué* subyacente de una determinada función y pueden suponer que es la única forma de resolver un problema concreto. En otras palabras, pueden (consciente o inconscientemente) llevarte a una copia del sistema antiguo en una nueva tecnología.

Es fundamental tener una perspectiva amplia. Elige la herramienta adecuada para resolver el problema real del cliente. Muchos ingenieros son demasiado optimistas. Sé realista y pragmático sin piedad. Sin embargo, la escalabilidad es importante: no todas las aplicaciones pueden representar un tercio del tráfico de Internet.

> *Hacer predicciones es muy difícil, especialmente si se trata del futuro.*
>
> —Proverbio danés

Aplica la regla de oro al software.

Es muy probable que, al principio de tu carrera educativa, hayas aprendido la regla de oro, que dice que debes tratar a los demás como te gustaría que te trataran a ti.[7] Este principio se aplica en todos los continentes y se remonta a miles de años atrás como un principio básico de cómo vivir en una sociedad civilizada. Pero resulta que no es solo un principio ético conciso. Puedes (¡y debes!) aplicar el mismo estándar a tu código.

Piensa en el desarrollador que seguirá tus pasos. Y, de nuevo, ¡ese desarrollador podrías ser tú mismo![8] ¿Qué *te* gustaría ver en el código base? ¿Qué mejoraría la vida del próximo desarrollador? En última instancia, el código es un mecanismo de comunicación. Sí, es una forma de indicar al ordenador que realice alguna tarea, pero eso palidece en importancia frente a su capacidad para comunicarse con otros desarrolladores. Escribe código que esté pensado para ser leído. Optimiza para los humanos, no para el compilador.[9]

Si quieres destacar entre la multitud, sigue la regla de oro. Escribe código limpio para que quienes te sigan puedan entenderlo y mantenerlo. Actualiza la documentación para evitar que otros pierdan el tiempo siguiendo un enfoque obsoleto. Crea diagramas claros que aclare en lugar de confundir. Dedica tiempo a responder preguntas y ayudar a los demás. Si juntas todo esto, serás muy solicitado; la gente querrá tenerte en sus equipos porque haces que todos a tu alrededor sean mejores.

Conclusión

La carrera de ingeniería de software es mucho más que mirar una pantalla y escribir código. Para ser más que un simple codificador o programador, es necesario hacer algo más que producir código sintácticamente correcto; se requiere un conjunto de habilidades bien equilibrado. No importa dónde o cómo hayas aprendido el oficio, ya sea a través de un programa de licenciatura en informática, un curso intensivo o materiales que hayas encontrado en Internet. Lo que importa es cómo combinas esas habilidades.

Lo que te enseñaron no es todo lo que necesitas saber para garantizar una carrera exitosa. En este capítulo se han compartido consejos sobre cómo evitar algunos de los errores más comunes, como buscar soluciones demasiado rápido, diseñar soluciones demasiado complejas y los peligros de los enfoques de fuerza bruta. Cuanto antes aprendas estas lecciones, mejor.

Ponerlo en práctica

Encontrar mentores puede ser increíblemente valioso al principio de tu carrera profesional. Busca en tu organización y pregunta educadamente a un ingeniero de software con más experiencia si podrías quedar con él para charlar tomando un café, que te invitarás tú. Habla con él sobre sus experiencias, lo que ha aprendido, lo que le hubiera gustado aprender antes, lo que le ha hecho más valioso en un equipo, lo que cree que tú deberías aprender, etc. También puedes ponerte en contac-

to con ponentes de eventos y autores. Muchos de ellos estarán encantados de ayudar a quienes los busquen.

La próxima vez que te asignen un error o una nueva función, haz una pausa antes de lanzarte inmediatamente al código. Asegúrate de que comprendes el problema que estás tratando de resolver. Dedica 30 minutos a investigar. ¿Existe alguna solución ya preparada que puedas aprovechar? Intenta esbozar o modelar conceptos en tu ámbito de problemas junto con posibles soluciones. Considera la posibilidad de discutir tus opciones con un compañero de trabajo con experiencia.

Echa un vistazo al índice de este libro. ¿En qué áreas temáticas te sientes más seguro? ¿En qué podrías mejorar? Elige un área temática sobre la que te gustaría aprender más y haz un plan para hacerlo. Pon en práctica las secciones «Poniéndolo en práctica» y lee los recursos adicionales de esos capítulos.

Tu marco de trabajo, biblioteca o proveedor de servicios en la nube pueden abstraer ciertas cosas de tu carga de trabajo diaria. Por ejemplo, aprovechar Spring Data te ahorrará escribir innumerables sentencias SQL. Si bien es absolutamente necesario que aproveches la productividad de estas abstracciones de grano más grueso, debes comprender la capa de abstracción *que se encuentra debajo de* la que estás utilizando. Tómate el tiempo para levantar el capó y echar un vistazo debajo; el conocimiento que adquieras te será de gran ayuda, especialmente cuando algo no funcione como estaba previsto.

Recursos adicionales

- *The Pragmatic Programmer: Your Journey to Mastery*, 20th Anniversary Edition, de Andrew Hunt y David Thomas (Addison-Wesley Professional, 2019)
- *El mítico mes-hombre: Ensayos sobre ingeniería de software*, edición aniversario, de Fred Brooks (Addison-Wesley, 1995)
- *Patrones de diseño: Elementos de software orientado a objetos reutilizable*, de Erich Gamma et al. (Addison-Wesley, 1994)
- *Practices of an Agile Developer*, de Venkat Subramaniam y Andy Hunt (The Pragmatic Bookshelf, 2006)
- *El programador productivo*, de Neal Ford (O'Reilly Media, 2008)
- *Ingeniería de Software at Google*, de Hyrum Wright et al. (O'Reilly Media, 2020)
- *The Staff Engineer's Path*, de Tanya Reilly (O'Reilly Media, 2022)
- «No Silver Bullet: Essence and Accidents of Ingeniería de Software», de Frederick P. Brooks Jr. (Departamento de Informática de la Universidad de Carolina del Norte en Chapel Hill, septiembre de 1986)
- *Code Complete*, 2.ª edición, por Steve McConnell (Cisco Press, 2004)

1 Por cierto, estos son los profesionales con más probabilidades de ser sustituidos por sistemas de codificación agenciales.

2 Como saber utilizar un depurador o estar familiarizado con los editores modernos.

[3] Aunque siempre vale la pena preguntarse por qué algo es como es, no te sorprendas si las conjeturas de tus colegas son tan acertadas como las tuyas.

[4] En pocas palabras, la notación O grande describe cómo algunos algoritmos son más eficientes que otros.

[5] Un glosario puede ser uno de los artefactos más impactantes y valiosos de un proyecto. Considera la posibilidad de crear uno si tu proyecto no lo tiene.

[6] Esto explica por qué tantas aplicaciones parecen reducirse al correo electrónico y las hojas de cálculo.

[7] O, más comúnmente, «Trata a los demás como te gustaría que te trataran a ti».

[8] Una vez más, todos los desarrolladores se han quedado mirando algún código preguntándose qué tonto lo habrá escrito, para luego darse cuenta poco a poco de que en realidad ellos mismos eran los tontos que lo habían escrito.

[9] Con la excepción de ciertos ejemplos de programación muy especializados. Lo sabrás cuando lo encuentres.